

**LAPORAN PROYEK ANTREAN TIKET BIOSKOP
ALGORITMA & STRUKTUR DATA**



Dosen Pengampu :

Dr. Shelvie Nidya Neyman S.Kom, M.Si.

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

Ditulis Oleh (Kelompok 4) :

1. M. Lutfi Ramadhan Warendra (J0403251040)
2. Fateeh Falah Hendarto (J0403251070)
3. Wildhan Dzikrihantara (J0403251098)

**MATA KULIAH ALGORITMA & STRUKTUR DATA
PROGRAM STUDI TEKNOLOGI REKAYASA
PERANGKAT LUNAK
SEKOLAH VOKASI
INSTITUT PERTANIAN BOGOR
BOGOR
2026**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.	
CARA PENGGUNAAN.....	1
1.1 Persyaratan Sistem.....	1
1.2 Langkah Instalasi.....	1
1.4 Menu Sistem Antrean.....	4
1.5 Contoh Alur Penggunaan Tipikal.....	5
1.6 Aturan Input.....	5
BAB II.	
PENJELASAN STRUKTUR DATA.....	6
2.1 Node.....	6
2.2 Queue (Antrian FIFO).....	7
2.3 Singly Linked List (Data Pemesanan).....	8
2.4 Struktur Data Pendukung.....	8
2.5 Struktur Data Field Pada Penyimpanan TXT (database).....	9
BAB III.	
VISUAL DIAGRAM ALUR.....	11
3.1 Diagram Alur Kerja Aplikasi Antrean Bioskop.....	11
3.2 Diagram Alur Aplikasi Antrian Bioskop.....	12
BAB IV.	
KENDALA & SOLUSI.....	13
BAB V.	
KESIMPULAN.....	17

BAB I.

CARA PENGGUNAAN

1.1 Persyaratan Sistem

Aplikasi ini memerlukan **Python 3.12** atau versi yang lebih baru. Daftar seluruh dependensi yang diperlukan dapat ditemukan pada berkas **requirements.txt**.

1.2 Langkah Instalasi

Sebelum menjalankan aplikasi Sistem Antrian Tiket Bioskop, pengguna perlu melakukan beberapa tahapan instalasi dan konfigurasi lingkungan kerja. Langkah-langkah instalasi yang dilakukan adalah sebagai berikut.

1. Mengunduh Source Code Proyek.

Source code proyek dapat diperoleh dengan melakukan proses cloning repositori GitHub menggunakan perintah berikut:

```
git clone https://github.com/wilanta/ASD_B_4.git
```

Alternatif lainnya, pengguna dapat mengunduh proyek dalam bentuk file ZIP melalui halaman repositori GitHub kemudian mengekstraknya ke direktori yang diinginkan.

https://github.com/wilanta/ASD_B_4

2. Masuk ke Direktori Project.

Setelah proses kloning atau ekstraksi selesai, pengguna perlu masuk ke direktori proyek menggunakan perintah:

```
cd ASD_B_4-master
```

Perintah tersebut akan memindahkan terminal ke folder utama proyek sehingga seluruh file dan dependensi dapat diakses dengan benar.

3. Membuat Virtual Environment.

Virtual Environment digunakan untuk mengisolasi dependensi proyek agar tidak mengganggu instalasi Python global maupun proyek lainnya.

Pembuatan Virtual Environment dilakukan dengan perintah:

```
python -m venv venv
```

Perintah tersebut akan membuat folder bernama venv yang berisi lingkungan Python terpisah khusus untuk proyek ini.

4. Mengaktifkan Virtual Environment.

Setelah Virtual Environment berhasil dibuat, langkah berikutnya adalah mengaktifkannya.

Pada Sistem Operasi Windows:

```
venv\Scripts\activate
```

Pada Git Bash (Windows):

```
source venv/Scripts/activate
```

Pada macOS atau Linux:

```
source venv/bin/activate
```

Apabila aktivasi berhasil, nama environment biasanya akan muncul di bagian awal terminal, misalnya:

(venv) C:\Users\User\antrian-tiket-bioskop>

5. Menginstal dependensi proyek.

Seluruh pustaka (library) yang dibutuhkan aplikasi telah dicantumkan pada file *requirements.txt*. Instalasi dependensi dilakukan menggunakan perintah berikut:

```
pip install -r requirements.txt
```

Proses ini akan menginstal seluruh package yang diperlukan agar aplikasi dapat berjalan dengan baik.

6. Menjalankan Aplikasi.

Setelah seluruh proses instalasi selesai, aplikasi dapat dijalankan menggunakan perintah berikut:

```
python main.py
```

Jika tidak terdapat kesalahan konfigurasi maupun dependensi, maka aplikasi Sistem Antrian Tiket Bioskop akan berjalan dan menampilkan menu utama pada terminal.

1.3 Navigasi Menu Utama

Setelah program dijalankan, tampilan menu utama akan muncul:

Pilihan	Keterangan
1	Masuk ke Sistem Informasi Antrean (memilih film terlebih dahulu)
2	Kelola Daftar Film (lihat, update, dan delete)

3	Tambah Film baru ke database
4	Buat Laporan Penjualan Tiket
0	Keluar dari program

1.4 Menu Sistem Antrean

Setelah memilih film, menu sistem antrean akan terbuka dengan fitur-fitur berikut:

Pilihan	Fitur	Keterangan
1	Masuk Antrean	Daftarkan nama penonton ke dalam antrian (enqueue)
2	Layani Antrean	Proses pemesanan kursi dan cetak invoice untuk penonton di depan antrian
3	Lihat Antrean	Tampilkan seluruh daftar antrian aktif
4	Lihat Data Pemesanan	Tampilkan daftar tiket yang sudah dipesan (Linked List)
5	Batalan Antrean	Hapus penonton tertentu dari antrian
6	Hapus Data Pemesanan	Hapus data tiket yang sudah dipesan dan kembalikan kursi
7	Cari Data Pemesanan	Cari data pemesanan berdasarkan nama
8	Reset	Kosongkan seluruh antrian dan data pemesanan pada <i>session</i> .
0	Kembali	Kembali ke menu utama

1.5 Contoh Alur Penggunaan Tipikal

- Jalankan program dengan python main.py
- Pilih [1] Sistem Informasi Antrean
- Pilih nomor film yang diinginkan
- Pilih [1] untuk mendaftarkan penonton pertama, masukkan nama
- Pilih [1] lagi untuk mendaftarkan penonton kedua, dan seterusnya
- Pilih [2] Layani Antrean, masukkan jumlah tiket (1-4), pilih kursi yang tersedia
- Invoice otomatis dicetak ke folder CRUD/history/invoices/
- Ulangi langkah sebelumnya untuk melayani antrean berikutnya

1.6 Aturan Input

- Nama penonton: hanya huruf alfabet, tidak boleh ada angka atau simbol.
- Jumlah tiket: angka 1-4 per transaksi, tidak boleh melebihi tiket yang tersedia.
- Nomor kursi: format K1, K2, ... Kn sesuai tampilan daftar kursi tersedia.

BAB II.

PENJELASAN STRUKTUR DATA

Sistem ini menggunakan dua struktur data utama yang dipilih secara cermat sesuai karakteristik masalah: Queue (Antrean) dan Singly Linked List. Keduanya diimplementasikan secara mandiri dari nol menggunakan kelas Node sebagai fondasi bersama.

2.1 Node

Seluruh struktur data dalam proyek ini dibangun di atas kelas Node yang didefinisikan di `CRUD/utils/node.py`. Setiap Node menyimpan :

Atribut	Tipe Data	Keterangan
customer_id	str	ID yang digenerate oleh fungsi idGenerator yang berupa 3 huruf dan 3 angka (misalnya: ABC123)
nama	str	Nama penonton (disimpan dalam lowercase)
jumlah_tiket	int	Jumlah tiket yang dipesan
nomor_kursi	list	Daftar nomor kursi yang dipilih
urutan_antrean	int	Nomor urut dalam antrian
judul_film	str	Judul film yang ditonton
create_at	str	Timestamp otomatis saat node dibuat
next	Node	Pointer ke node berikutnya

2.2 Queue (Antrian FIFO)

Diimplementasikan di `CRUD/operations/queue.py`

Alasan Pemilihan: Queue dipilih untuk mengelola antrian penonton karena sifatnya yang FIFO (First In, First Out), siapa yang lebih dulu mendaftar, akan lebih dulu dilayani. Ini secara alami mencerminkan sistem antrian dunia nyata di loket bioskop.

Queue diimplementasikan sebagai Singly Linked List dengan dua pointer ujung:

- front (kepala): penonton yang sedang antri paling depan dan akan segera dilayani
- rear (ekor): penonton yang paling baru bergabung ke antrian

Method	Kompleksitas	Fungsi
enqueue(nama)	O(1)	Tambahkan penonton ke bagian rear antrian
dequeue()	O(1)	Hapus penonton terdepan (front) setelah dilayani
isEmpty()	O(1)	Cek apakah antrian kosong
showQueue()	O(n)	Tampilkan seluruh isi antrian
cancelQueue(nama)	O(n)	Batalkan antrian penonton tertentu (tengah-tengah)
updateQueue(...)	O(1)	Perbarui data tiket pada node front
adjustAntrean(node)	O(n)	Perbarui ulang nomor urut setelah pembatalan

2.3 Singly Linked List (Data Pemesanan)

Diimplementasikan di `CRUD/operations/ticket.py`

Alasan Pemilihan: Linked List dipilih untuk menyimpan data pemesanan karena ukurannya yang dinamis — data bisa bertambah atau berkurang sewaktu-waktu tanpa perlu mengalokasikan memori secara statis seperti array. Selain itu, penghapusan data di posisi mana pun lebih efisien secara semantik karena hanya perlu mengubah pointer.

Method	Kompleksitas	Fungsi
<code>addTicket(...)</code>	$O(1)$	Tambahkan node pemesanan baru ke bagian tail
<code>showTickets()</code>	$O(n)$	Tampilkan seluruh data pemesanan
<code>deleteTicket(nama)</code>	$O(n)$	Hapus node pemesanan dan kembalikan tiket
<code>searchTicket(nama)</code>	$O(n)$	Cari data pemesanan berdasarkan nama
<code>isEmpty()</code>	$O(1)$	Cek apakah linked list kosong

2.4 Struktur Data Pendukung

Struktur	Lokasi	Kegunaan
Dictionary (dict)	<code>dataOps.py</code>	Penyimpanan data film dan log pemesanan dalam format JSON di file <code>.txt</code>
List (Python)	<code>order.py</code>	Menyimpan daftar kursi yang tersedia (<code>available_seat</code>)
<code>sorted()</code>	<code>seatSort.py</code>	Mengurutkan nomor kursi secara numerik (K1, K2, ..., K10) bukan alfabetis

2.5 Struktur Data Field Pada Penyimpanan TXT (database)

2.5.1 Data film

Nama field	Tipe data
film_id	str
judul_film	str
kuota_penonton	int

2.5.2 Log pemesanan

Nama field	Tipe data
log_id	str
nama	str
jumlah_tiket	int
urutan_antrean	int
judul	str
date	date
nomor_kursi	list

2.5.3 Temp log pemesanan

Nama field	Tipe data
log_id	str
nama	str

jumlah_tiket	int
urutan_antrean	int
judul	str
date	date
nomor_kursi	list

2.5.4 Data antrean

Nama field	Tipe data
type	str
film_id	str
urutan_counter	int
nodes	list

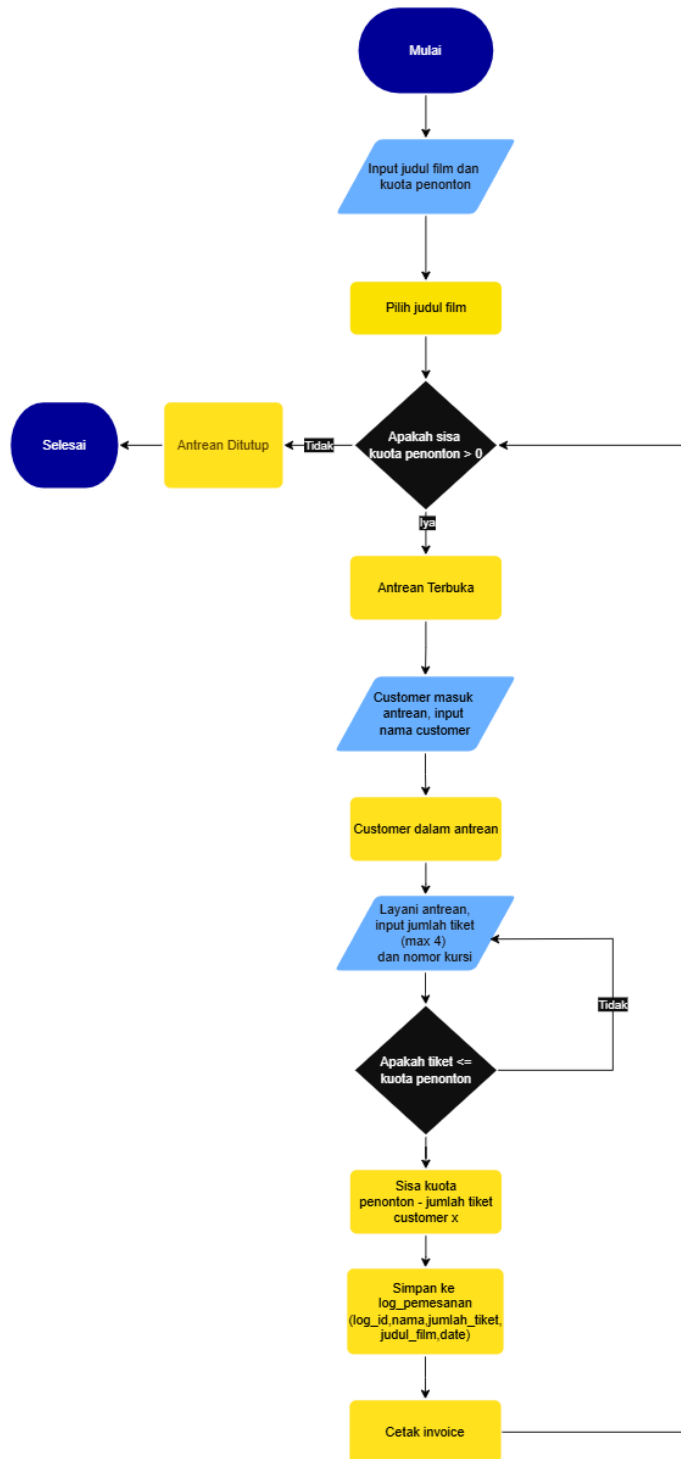
2.5.5 Temp seat

Nama field	Tipe data
film_id	str
judul_film	int
nodes	list

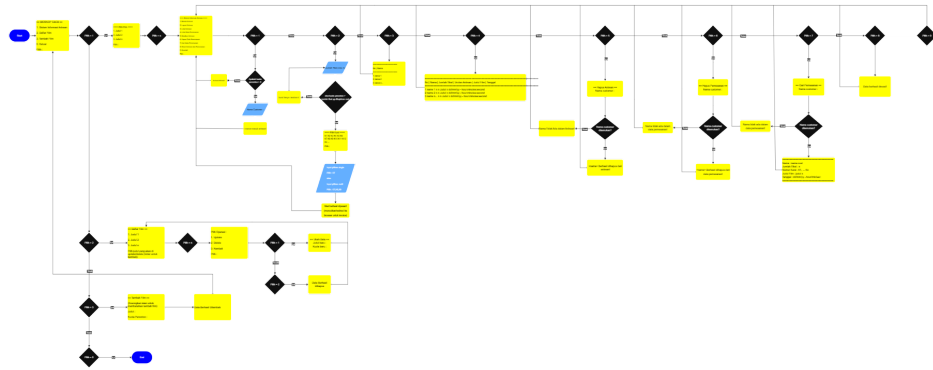
BAB III.

VISUAL DIAGRAM ALUR

3.1 Diagram Alur Kerja Aplikasi Antrean Bioskop



3.2 Diagram Alur Aplikasi Antrian Bioskop



Sumber:

- [Sumber Diagram](#)
- https://miro.com/app/board/uXjVGD2OHVo=?share_link_id=12187728476

BAB IV.

KENDALA & SOLUSI

4.1 Kendala 1: Pembatalan Antrian di Tengah (cancelQueue)

Masalah: Ketika ada dua atau lebih penonton dengan nama yang sama dalam antrian, sistem harus bisa membedakan siapa yang ingin dibatalkan. Kesalahan pertama adalah menggunakan nama saja sebagai kunci pencarian — ini menyebabkan program selalu menghapus penonton pertama dengan nama tersebut, bukan yang dimaksud pengguna.

Solusi: Menambahkan method `countNameInQueue()` sebagai validator - jika terdapat lebih dari satu nama duplikat, sistem akan menampilkan daftar beserta nomor urut antreannya dan meminta pengguna memilih urutan mana yang ingin dibatalkan. Selain itu, ditambahkan method `adjustAntrean()` untuk merapikan kembali nomor urut seluruh node setelah salah satu node dihapus.

Potongan logika solusinya:

```
count_nama = self.countNameInQueue(nama)
if count_nama > 1:
    # Tampilkan semua urutan antrean dengan nama sama
    # Minta user pilih urutan yang ingin dibatalkan
    self.adjustAntrean(node_setelah_yang_dihapus)
```

4.2 Kendala 2: Pengurutan Kursi Setelah Pengembalian (seat_sort)

Masalah: Ketika kursi dikembalikan setelah pembatalan pemesanan dan ditambahkan kembali ke list `available_seat` menggunakan `.extend()`, urutan kursi menjadi berantakan. Misalnya K10 muncul

sebelum K2 karena Python mengurutkan string secara leksikografis ('K10' < 'K2' dalam urutan string).

Solusi: Dibuat fungsi `seat_sort()` di `CRUD/utlis/seatSort.py` yang menggunakan `sorted()` dengan key berupa lambda yang mengekstrak angka dari string kursi (misal 'K12' → 12) dan mengurutkan secara numerik. Ini memastikan urutan selalu K1, K2, K3, ..., K10, K11 dan bukan K1, K10, K11, K2.

Potongan logika solusinya:

```
def seat_sort(seat_list: list):  
    return sorted(  
        seat_list,  
        key=lambda x: int("".join(filter(str.isdigit, x)))  
    )
```

4..3 Kendala 3: Sinkronisasi Queue dan Linked List

Masalah: Saat penghapusan data pemesanan (Linked List) dilakukan, tiket dan kursi yang dipesan harus dikembalikan ke `available_ticket` dan `available_seat`. Awalnya pengembalian ini tidak diimplementasikan, sehingga kursi yang sudah dipesan tetap terkunci meskipun pemesanannya sudah dihapus.

Solusi: Saat penghapusan data pemesanan (Linked List) dilakukan, tiket dan kursi yang dipesan harus dikembalikan ke `available_ticket` dan `available_seat`. Awalnya pengembalian ini tidak diimplementasikan, sehingga kursi yang sudah dipesan tetap terkunci meskipun pemesanannya sudah dihapus.

Potongan logika solusinya:

```
refunded_ticket, refunded_seat = ll.deleteTicket(nama)
```



```

available_ticket += refunded_ticket
available_seat.extend(refunded_seat)
available_seat = seat_sort(available_seat)

```

4.4 Kendala 4: Validasi Input Nama Penonton

Masalah: Nama dengan spasi (seperti 'Budi Santoso') tidak lolos validasi `isalpha()` karena spasi bukan karakter alfabet. Ini mengakibatkan nama dua kata tidak bisa didaftarkan sama sekali.

Solusi: Kondisi validasi diubah menjadi `nama.strip().replace(' ', '').isalpha()` — menghapus spasi terlebih dahulu sebelum memeriksa karakter alfabet, sehingga nama dengan spasi dapat diterima, sementara angka dan simbol tetap ditolak.

Potongan logika solusinya:

```

while not nama_penonton.strip().replace(' ', '').isalpha():
    print('Nama harus berupa huruf saja.')
nama_penonton = input('Masukkan nama: ').strip()

```

4.5 Kendala 5: State management untuk antrean

Masalah: Program tidak akan save antrean yang ada di suatu film. Jika pengguna keluar dari sistem antrean, maka semua antrean akan hilang.

Solusi: Implementasi operasi pada memory dengan dictionary khusus sistem antrean. Bisa dikembangkan menjadi DB table sendiri.

Ringkasan Kendala

No	Kendala	Root Cause	Status
----	---------	------------	--------

1	cancelQueue dengan nama duplikat	Pencarian hanya berdasarkan nama	Selesai
2	Urutan kursi kacau setelah refund	Pengurutan string leksikografis	Selesai
3	Kursi tidak kembali saat hapus pesanan	deleteTicket tidak mengembalikan data	Selesai
4	Nama dua kata ditolak validasi	isalpha() tidak mengizinkan spasi	Selesai
5	Data antrian tidak tersimpan	Data tidak disimpan dalam variabel yang persistent	Selesai

BAB V.

KESIMPULAN

Proyek Sistem Antrean Tiket Bioskop ini berhasil mengintegrasikan konsep struktur data Queue dan Singly Linked List ke dalam aplikasi fungsional berbasis Python. Penggunaan Queue menjamin keadilan dalam pelayanan penonton (FIFO), sementara Linked List memberikan fleksibilitas dalam pengelolaan riwayat pemesanan yang bersifat dinamis.

Melalui pengembangan ini, ditemukan bahwa tantangan utama dalam struktur data manual bukan hanya pada penyimpanan, melainkan pada integritas data saat terjadi perubahan (seperti pembatalan atau penghapusan). Penggunaan logika pemrosesan string kustom dan manajemen pointer yang teliti terbukti mampu mengatasi kendala teknis tersebut, menghasilkan sistem yang robust dan siap digunakan untuk simulasi operasional bioskop.